

Reviewer Pack — Eigenvalue Count and SOS Degree for Max-CUT Approximation

Entry c0336452dd8d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 06:56:25 UTC

Eigenvalue Count and SOS Degree for Max-CUT Approximation

Entry ID: c0336452dd8d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 06:56:25 UTC

1. Conjecture

Field A (mathematical branch): Random Matrix Theory **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For a random graph $G(n, 1/2)$, the minimal SOS degree required to approximate max-CUT within a factor of $0.878 + \varepsilon$ is $\Omega(\sqrt{n})$, where the constant depends on the number of eigenvalues of G 's adjacency matrix in the interval $[-\sqrt{(n \log n)}, \sqrt{(n \log n)}]$.

Rationale (proposer's reasoning):

The eigenvalue distribution of random graphs influences the structure of the feasible region for SOS relaxations. Concentration of measure in random matrices suggests that eigenvalues cluster around certain values, which could necessitate higher SOS degrees to capture the necessary precision for approximation.

Taxonomy category: SOS_DEGREE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c43157d995f823e0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A):
    n = len(A)
    augmented_matrix = [A[i] + [0] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
        factor_i = 1 / augmented_matrix[i][i]
        for j in range(n):
            augmented_matrix[i][j] *= factor_i
        for k in range(n):
            if k != i:
                factor_j = augmented_matrix[k][i]
                for j in range(n):
                    augmented_matrix[k][j] -= factor_j * augmented_matrix[i][j]
    eigenvalues = [augmented_matrix[i][i] for i in range(n)]
    return eigenvalues

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    adjacency_matrix = [[random.choice([0, 1]) if i != j else 0 for j in range(n)] for i in range(n)]

    # Calculate eigenvalues
    eigenvalues = gaussian_elimination(adjacency_matrix)
    eigenvalue_count = sum(1 for ev in eigenvalues if -math.sqrt(n * math.log(n)) <= ev <= math.sqrt(n * math.log(n)))

    # Approximate max-CUT using a simple heuristic (e.g., random partitioning)
    cut_value = 0
    for i in range(n):
        for j in range(i+1, n):
            if adjacency_matrix[i][j] == 1:
                if random.choice([True, False]):
                    cut_value += 1
```

```

# Minimal SOS degree heuristic (simplified)
sos_degree = int(math.sqrt(n))

return {
    "metric_name": "SOS Degree",
    "metric_value": sos_degree,
    "instances_tested": 1,
    "conjecture_holds": eigenvalue_count >= math.sqrt(n),
    "counterexample": "" if eigenvalue_count >= math.sqrt(n) else f"Eigenvalue count {eigenval
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
        print(f"RESULT: FALSIFIED counterexample=\"Eigenvalue count < sqrt(n)\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'SOS Degree', 'metric_value': 5, 'instances_tested': 1, 'conjecture_holds': Tr
TRIAL: {'metric_name': 'SOS Degree', 'metric_value': 4, 'instances_tested': 1, 'conjecture_holds': Tr
TRIAL: {'metric_name': 'SOS Degree', 'metric_value': 3, 'instances_tested': 1, 'conjecture_holds': Tr
TRIAL: {'metric_name': 'SOS Degree', 'metric_value': 4, 'instances_tested': 1, 'conjecture_holds': Tr
TRIAL: {'metric_name': 'SOS Degree', 'metric_value': 5, 'instances_tested': 1, 'conjecture_holds': Tr
TRIAL: {'metric_name': 'SOS Degree', 'metric_value': 3, 'instances_tested': 1, 'conjecture_holds': Tr
TRIAL: {'metric_name': 'SOS Degree', 'metric_value': 5, 'instances_tested': 1, 'conjecture_holds': Tr

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71ec784e.py", line 71, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71ec784e.py", line 43, in run_trial
    eigenvalues = gaussian_elimination(adjacency_matrix)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71ec784e.py", line 26, in gaussian_elim

```

```
factor_i = 1 / augmented_matrix[i][i]
           ~^~~~~~
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing reliable evaluation of the conjecture's validity across sufficient instances. | next: Fix the gaussian_elimination function to handle singular matrices, then re-run tests with more seeds to validate the conjecture

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	82648
2	propose	ollama_remote	qwen3:8b	0	0	112280
3	preregistration	ollama_remote	qwen3:8b	0	0	31698
4	novelty	ollama_remote	qwen3:8b	0	0	31651
5	novelty	ollama_remote	qwen3:8b	0	0	21209
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40695
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8882
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9537
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10564
10	judge	ollama_remote	qwen3:8b	0	0	23547

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 372712 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/c0336452dd8d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c0336452dd8d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c0336452dd8d.tar.gz` (if generated)